Lux Gives Terminal-Hosted AI Coding Agents a Native, Local-First Visual Surface, No Vendor Compute

Agents describe UI as a typed JSON element tree; Lux paints it with GPU-accelerated Dear ImGui in a persistent shared window on the developer's own machine

Today the agent shows me complex data and visualizations; next, I drive a terminal-based agent through the GUI itself. The element set is fixed and curated, but the ways elements compose are unlimited

Press Release

AUSTIN, TX, January 2027. Punt Labs today released Lux, a local-first visual output surface for terminal-hosted AI coding agents. The most capable agentic coding tools, Claude Code[1] and Codex[2], run in the terminal on the developer's own machine, but their structured output is trapped in scrollback: a table, a dependency diagram, or a metrics dashboard arrives as monospace text or an ad hoc screenshot. Lux gives those agents a screen. An agent describes what it wants as a tree of typed JSON elements; Lux renders it with Dear ImGui[3] in a persistent, GPU-accelerated window, entirely on the developer's machine, with no vendor cloud. Unlike web dashboards that require running the agent on vendor compute, or the terminal that has no structure at all, Lux keeps the agent local and gives its output a surface the developer can read, filter, and click.

Problem

Developers now run several coding agents at once, across repositories, sessions, and machines. Tracking what each one is doing is hands-on and error-prone, and it compounds with every agent added. The most capable agents, Claude Code and Codex, are terminal-hosted, with full tool use, MCP integration, and multi-turn autonomous operation. That power comes with no visual surface. When an agent produces a table of failing tests, a diagram of a module graph, or a dashboard of build status, the developer reads it as raw text or scrolls back to find it. There is no way to filter the table, select a row, or keep the view live as the underlying data changes.

Vendors are answering with hosted web dashboards that often require running the *entire agent* (not just the model call) on the vendor's compute: the developer trades local control for a viewport. IDE-hosted tools and MCP-host apps solve pieces of this but not the whole problem: one manages agents with text-only output, the other renders rich interfaces but cannot drive a terminal agent's autonomous, multi-turn workflow end-to-end (see FAQ 2). The result is a split: the tools with the most agentic power have no visual output, and the tools with visual output lack the agentic power.

Solution

Lux is a persistent local service that agents write to over MCP. An agent calls a tool such as `show_table` or `show_dashboard` with structured content; the display renders it in a native GPU-accelerated window using Dear ImGui. Multiple agents share the one window, each in its own frame with clear attribution, so a developer glances at a single surface and sees what every session is doing. Everything runs on the developer's machine: the Hub holds authoritative UI state, the display renders a replica, and no data leaves the box.

Lux is not a passive canvas. It ships two capabilities that matter in daily work. First, a rich data-display core: filterable, searchable tables with row selection and detail panels; dashboards that compose metric cards, charts, and tables; diagrams and images. The element vocabulary is

deliberately limited: a curated, fixed set of built-in kinds that compose without limit, the same way a small set of ImGui widgets does. Second, and the direction Lux is heading, an ask-the-user interaction loop: an agent can pose a question through a dialog, checkbox, or button and receive the answer back. When the user clicks, the interaction routes back to the owning Hub, the real handler runs there, and the Hub re-sends the updated view. That loop is the first interaction primitive, the seed of a two-way surface where the user drives a terminal-based agent through a GUI instead of only reading its output. The canonical example is the beads issue browser: a filterable table that auto-refreshes on every `bd` command through a post-tool hook, running live in every active agent session (see FAQ 8).

Customer Quote

> *"I keep four Claude Code sessions open across three repos. Before Lux I had four terminals and no idea what any of them were doing without scrolling back. Now one window shows me a beads board, a test-run table, and a build dashboard, one frame per agent, all live. When a migration script needed me to confirm a destructive step, the agent didn't dump a wall of text and wait; it put a dialog on the screen with the two options. I clicked. That is the difference between watching a transcript and using a tool."*
>
> — **Maya Torres**, Staff Engineer at a Series B DevTools Startup

Getting Started

Getting started takes one command:

1. Install:
   ```
   curl -fsSL https://raw.githubusercontent.com/punt-labs/lux/main/install.sh | sh
   ```
2. Restart Claude Code. The Lux display window opens automatically the first time any agent sends visual output.
3. Ask an agent to show you something: "show me the beads board," "put the test results in a table." The view appears in the window, live.

No API key is required. Lux runs on macOS and Linux, needs Python 3.13+, and installs the `punt-lux` package plus the Claude Code plugin.

Spokesperson Quote

> "The most powerful coding agents live in the terminal, and the terminal gives them one output mode: text. We built Lux so an agent can hand the developer a real surface (a table you filter, a dashboard you scan, a dialog you click) without moving the agent onto someone's cloud. The part I care most about is the interaction loop: the agent asks, the user answers, and the answer flows back to the handler that asked. That turns a display into a conversation. And because the whole thing is local and native, 'Lux is showing me stale data' is not a failure mode: the Hub owns the state and re-sends the view when it changes."
>
> — **Jim Freeman**, Punt Labs

Call to Action

Lux is free and open-source under the MIT license. Visit `https://github.com/punt-labs/lux` to install, read the target architecture, or contribute a new element kind.

Frequently Asked Questions

External FAQs

Q1: What is Lux and who is it for?

Lux is a visual output surface for terminal-hosted AI coding agents. It is a persistent, GPU-accelerated window, built on Dear ImGui[3], that agents write to and read from. It is for developers who run terminal agents such as Claude Code[1] or Codex[2] and want their structured output as something they can see and act on: filterable tables, dashboards, diagrams, and interactive prompts, rather than scrollback text. The element kinds are fixed and curated; they compose without limit, covering any layout the way a small set of ImGui widgets does. Everything renders and runs on the developer's own machine. Lux runs on macOS and Linux.

Q2: How is this different from terminal scrollback, Claude Desktop, or vendor dashboards?

Terminal agents (Claude Code, Codex) are the most capable but output only text. Claude Desktop renders rich interfaces and is itself an MCP host, but it cannot drive a coding session end-to-end: it lacks the terminal agent's autonomous, filesystem-level, multi-turn tool-driven workflow. Vendor web dashboards provide visuals but often require running the whole agent on vendor compute. IDEs such as Cursor manage agents with text-only output.
Lux fills the gap: a native visual surface for terminal-hosted agents, running on the developer's machine. It is not a chat interface and not a cloud console. It is a screen the agents share: one window, one frame per agent, live. See FAQ 9 for the detailed competitive analysis.

Q3: How do I get started?

One command installs Lux and registers the Claude Code plugin. Restart Claude Code. The display window opens by itself the first time an agent sends visual output. There is no separate app to launch and no configuration to edit. Ask an agent to show you a table or a board and it appears.

Q4: How much does it cost?

Free and open-source under the MIT license. There is no paid tier, no account, and no API key requirement for the display itself. Lux consumes the same agent you already run; it adds no per-call cost of its own.

Q5: What happens to my data?

Everything stays local. Agents talk to the Hub over MCP; the Hub talks to the display over a Unix domain socket under `/tmp/lux-<user>/`. Per-project configuration lives in your repository at `<repo>/.punt-labs/lux.md`. There is no telemetry and no cloud dependency: nothing about your code, data, or agents' output leaves the machine. The Hub owns authoritative UI state and the display renders a replica; render calls never cross the wire, only UI state does.

Q6: What can Lux render today, and can agents ask me questions?

Two things ship and are in daily use. The data-display core covers filterable and searchable tables with row selection and detail panels, dashboards (metric cards, charts, tables), diagrams, and images: 25 element kinds in all. The interaction loop lets an agent ask you a question through the UI: a dialog, a checkbox, or a button, with your answer routed back to the agent's handler. That ask-the-user loop shipped with the Hub/Display interaction model. Completing coverage of every ImGui widget as a built-in element is the v2 direction (see FAQ 14).

Internal FAQs

Value & Market

Q7: What is the total addressable market?

The primary market is developers running terminal-hosted AI coding agents: Claude Code[1] and Codex are the leading tools, both terminal-hosted. That population is a subset of professional developers, expanding as agentic coding moves from novelty to daily practice. We decline a precise headcount: no reliable public figure for active terminal-agent developers exists to cite without inventing it. As an order-of-magnitude *assumption* only (not a measurement), the cohort is plausibly 10^5–10^6 today, a small fraction of the world's professional developers, and growing. Positioning, not sizing, is the point. Lux is free and MIT-licensed; its value to Punt Labs is ecosystem pull, not license revenue (see FAQ 17). The relevant question is not "how many will pay" but "how many terminal-agent developers would keep a local visual surface open once they have one." That is the hypothesis current internal use supports and external use has not yet tested.

Q8: What evidence do we have that customers want this?

Lux is released (v0.19.1 on PyPI as `punt-lux` and on the plugin marketplace) and used daily. The precise scope matters and we state it plainly: the daily user is *one* primary human developer (Jim Freeman) plus the agent fleet he runs across the Punt Labs repositories. When we say the beads issue browser (a filterable table that auto-refreshes on every `bd` command through a post-tool hook) "runs in every session," that counts *agent* sessions, not distinct human users. It is a genuine daily-driver signal, but n=1 human. That single view still demonstrates the two claims the product rests on: the data-display core has real daily value, and the live-refresh hook makes the surface feel current without the agent having to coordinate updates. The interaction loop, an agent asking a question and receiving the answer through a dialog, is exercised in real sessions, not just tests.

The honest gap: this evidence is *internal*. The target customer (an external developer running several terminal agents at once, the profile in the press-release quote) has not used Lux yet. Punt Labs is the current *user* and proof point, not the market. External adoption is zero: no usage data, no retention figures, no third-party testimonials. Validation confined to the company that built the product is, in the four-risks framing, a High Value-risk signal, and this document rates it as such. Every "value" claim here should be read against that boundary (see FAQ 10 and the Value risk in the Risk Assessment).

Q9: Who are the competitors and why will we win?

The landscape divides on two axes: agentic power and visual surface.

- **Claude Code / Codex.** Terminal-hosted, the most capable agentic tools: full tool use, MCP, multi-turn autonomy. No visual output beyond scrollback. *These are Lux's integration targets, not rivals.*
- **Claude Desktop.** Renders novel visual interfaces and is the canonical MCP host, but lacks the terminal agent's autonomous, filesystem-level, multi-turn workflow end-to-end.
- **Cursor.** IDE with strong agent management, but structured output stays text. Extensible via VS Code extensions, which are IDE-scoped and hand-written.
- **Vendor web dashboards.** Provide visual management, but often require running the whole agent on vendor compute. The developer gives up local control for a viewport.

- **Jupyter.** Rich visual output via the display protocol, but notebook-first, not agent-first.

Primary threat: Anthropic-native visual output. Anthropic has every incentive to build visual panels into Claude Code. A basic built-in display would narrow Lux's advantage to "native GPU-accelerated and fully local vs. vendor-native." Survival rests on three structural facts a competitor cannot copy cheaply: Lux renders natively with ImGui on the developer's GPU and keeps the agent local; Lux is a shared surface across *multiple* agents and tools, not a panel bolted onto one vendor's client; and the Hub/Display split lets an agent on a remote box drive a GUI on your local screen, because UI state rather than render calls crosses the wire (see FAQ 12). A web-native competitor would have to abandon web-only rendering to match the local-native path: a platform decision they are unlikely to reverse.

Q10: What is the next step to validate external demand?

The riskiest assumption is that Lux's internal value transfers to developers outside Punt Labs. The first external-validation experiment is to put Lux in front of external developers who run 2+ concurrent agent sessions daily and measure whether they keep the window open past the first day. Concretely: recruit a cohort, ship them the one-command install, and instrument nothing on their machine (no telemetry by design); instead, interview at day 7. The decision threshold: if fewer than half describe the display as something they would miss if it were removed, revisit the positioning before investing in the v2 element-coverage buildout.

Q11: How do external developers discover an MIT-licensed tool?

Three concrete channels, none of them yet proven for Lux.

- **The Claude Code plugin marketplace.** Lux is published there; a developer browsing plugins can find and install it. This is the most direct channel and requires no separate discovery of Punt Labs.
- **GitHub.** The repository is public and MIT-licensed; the one-command install lives in the README. This is organic search and word-of-mouth: real, but slow and unmeasured.
- **Cross-surfacing from the companion suite.** A developer who adopts vox, biff, or quarry encounters Lux as the surface those tools render into, and vice versa. The suite is the distribution flywheel the ecosystem thesis depends on (see FAQ 17).

The honest position: distribution is currently a hope, not a proven motion. "Organic and word-of-mouth" is not a strategy, and we have no acquisition data. The marketplace listing and the suite cross-surfacing are the two levers we can pull; whether either converts external installs is exactly what the day-7 cohort (see FAQ 10) begins to answer.

Technical

Q12: What is the architecture?

Lux is a three-tier distributed Hub/Display system. The target design lives in `docs/architecture/target/target.md`.

- **`luxd`**: the session Hub. It decodes wire elements, holds authoritative UI state, ownership, and handler dispatch in `HubDisplay`, pushes element replicas to the display, and re-dispatches interactions that come back from clicks.
- **`lux-display`**: the ImGui renderer. It stores a full copy of the UI, renders it every frame, and wraps handlers so that a click forwards back to the owning Hub instead of executing locally.

- **`mcp-proxy`**: the transport bridge from Claude Code's stdio to the `luxd` WebSocket.

MCP is a first-class front door into the Hub, not a thin adapter. The boundary rule is strict: UI state crosses the wire (scene trees, serialized element objects, interaction messages); render calls do not. Interactive elements use two-tier handler dispatch (the Hub owns the real handler and the Display forwards the interaction back), so the Hub and Display can be separate processes, potentially on separate machines.

This lets the three tiers sit on different machines: today the Display connects to the Hub over a Unix socket, and agents connect to the Hub over a WebSocket bound to localhost (127.0.0.1:8430, localhost-only). A remote agent, for example one running over SSH on another host, reaches the Hub today by tunneling that localhost port. Native TCP, so a remote agent connects to the Hub directly with no tunnel, is planned and not yet shipped. The practical payoff: your most powerful agent can run on a remote box and still drive a GUI on your local screen, because only UI state crosses the wire.

Q13: What makes the engineering distinctive?

The display-process concurrency is formally specified in Z and model-checked with ProB. The model-check covers only the display singleton spawn/bind/reap *lifecycle*, not the rendering, protocol, or handler-dispatch surface: it is not a claim that the whole system is formally verified. Multiple agents may simultaneously decide no display is running and race to become the one process that owns a single Unix-socket path. Exactly one must win. That spawn/reap/bind singleton lifecycle is modeled in `docs/display_lifecycle.tex` as a finite state machine over concurrent agents, type-checked `fuzz`-clean, and exhaustively model-checked. ProB enumerates every interleaving and confirms five invariants: singleton-serving (never two live owners), never-unlink-live (a probe cannot delete a socket whose owner is alive), no-two-winners across the bind window, lost-racer-clean-exit (the process that loses the race exits before opening a window), and deadlock-freedom under the two-lock spawn/bind discipline.

This matters because the failure it prevents is exactly the one that bit us in testing: a second agent probing during the window between `bind` and `listen` reads the fresh socket as dead, unlinks it, and opens a second window. Proving the interleaving space exhaustively, rather than testing a few orderings, is rare rigor for a developer tool, and the specification is a regression artifact for any future lifecycle change.

Q14: What is the v1-to-v2 roadmap, and what is the migration risk?

v1 (shipped): "my agent can show me complex data and visualizations." The rich data-display core, plus the ask-the-user loop (dialog, checkbox, button) as the first interaction primitive, on the Hub/Display model.

v2 (direction): "I can use a GUI to interact with my agent." The surface turns two-way, so the user drives a terminal-based agent through a GUI instead of only viewing its output. Completing ImGui widget coverage is the means to that end, not the end in itself: richer interaction needs more input primitives (sliders, inputs, selects) and the composites that arrange them. This is still Punt Labs building one fixed, curated vocabulary, explicitly *not* agents authoring arbitrary extensions and not a plugin registry, because a bounded element set already composes without limit.

The honest risk is the migration itself. Lux is mid-rewrite from an older single-process design onto a distributed Element-ABC path with two-tier remote handler dispatch. Four of the 25 element kinds (text, button, checkbox, and dialog) are on the new Element-ABC path today; the other 21 are legacy dataclasses being migrated family by family into curated built-ins. The

sequencing, per-kind contract, and open design decisions are mapped in `docs/architecture/element-migration-audit.md`. The scope is real work (interactive inputs need a handler-wrapping seam, composites need typed children, the table kind carries built-in view state), and it is the primary feasibility risk (see the Risk Assessment). It is mitigated by incremental crossing: each kind flips independently and mixed scenes keep working, with the legacy path deleted only after the last kind crosses.

Q15: What is the development approach and delivery order?

Delivery is ordered by dependency, not by calendar. The pattern is established on the lowest-risk element kinds first and then climbs interactivity and composite complexity:

1. Reconcile the four migrated exemplars into one source of truth.
2. Invert the handler-wrapping seam so each interactive element declares its own interaction (prerequisite for migrating the input kinds).
3. Basics display-only leaves (image, separator, progress, spinner, markdown).
4. Interactive value inputs (slider, combo, text/number inputs, radio, color picker, selectable).
5. Simple composites (group, tab bar, collapsing header).
6. Stateful composites (window, modal).
7. Draw and plot.
8. Table, on its own (it carries built-in filter/selection state).
9. Retire the legacy render path once every kind has crossed.

What gets cut if scope compresses: v2 element coverage is a fast-follow, not a v1 blocker. The data-display core and the interaction loop already ship; the migration improves internal structure and widens the built-in vocabulary without gating the value already in daily use.

Q16: If Lux succeeds, what breaks first?

Three known limits, stated with their current status rather than reassurance.

1. **The image texture cache grows without eviction.** Every image an agent shows uploads a texture that is never freed: an unbounded memory leak under sustained image use. Status: known and tracked only as a Should-Do (a texture-cache eviction policy, Feature 14); it is a real limitation today, not a solved problem. It is the first thing that breaks under a long-running session that renders many distinct images.
2. **Renderer maintainability debt.** `display/server.py` is roughly 1,550 lines, well over the 300-line target, and `element_renderer.py` is also over. Decomposition is in progress (the original monolith was already split once). This does not break at runtime, but it slows every rendering change and is why the render layer is undertested. Status: actively being decomposed, incrementally.
3. **Many-agent contention on one window.** The shared window is a single surface; a large agent fleet all pushing scenes contends for one display and one Hub. The whole-tree resend replication policy is simple but re-sends the whole affected UI on change. Status: adequate at the current fleet size; a heavy multi-agent load is untested and would be the first place the simple replication model needs a diff protocol.

Business

Q17: What is the revenue model?

Lux has no direct revenue. It is free and MIT-licensed. Its strategic value is ecosystem pull for the Punt Labs agent-tool suite. Lux is the visual counterpart to the other tools in the suite: vox (voice), biff (presence), and quarry (search). It surfaces them, too: the beads board, presence panels, and status views render in the Lux window. A developer who adopts Lux is one step from the rest of the suite. On the cost side there is no COGS (Lux runs entirely on the developer's own machine), so Punt Labs' development and maintenance time is the only real expense. This is an infrastructure and ecosystem play, acceptable because it is treated as a public good with strategic return, not a profit center. If Lux does not pull adoption of the companion tools, the investment is a public good without business return: that is the viability risk.

Q18: What are the key metrics for success?

1. **External retention**: of developers who install Lux outside Punt Labs, the share still running it after one week. This is the metric that converts the internal-only evidence into external validation.
2. **Companion-tool pull**: share of Lux installers who adopt at least one other Punt Labs tool (biff, vox, quarry). This measures the ecosystem thesis directly.
3. **Element-coverage progress**: element kinds migrated onto the Element-ABC path (4 of 25 today), tracking the v2 buildout.
4. **Interaction usage**: sessions that use the ask-the-user loop, distinguishing Lux-as-display from Lux-as-interaction-surface.

Decision threshold: if external one-week retention stays near zero after a genuine external trial, the product is an internal tool, not a market one, and the roadmap should stop treating external adoption as a near-term goal.

Q19: Why now? What has changed?

Three things converged. First, terminal-hosted agents, Claude Code and Codex, became the most capable agentic coding tools, and developers now run several at once, all producing output invisible beyond scrollback. Second, the vendor response is web dashboards that pull the agent onto remote compute, opening a clear gap for a local-first visual surface. Third, Lux itself proved the display and the interaction loop have real daily value internally, and the Hub/Display architecture that makes the shared, attributed, multi-agent window possible is shipped, not speculative.

Q20: What are we explicitly not building?

Lux is a curated visual surface, not a platform for arbitrary code. It will not become a self-extending display where agents author and register their own extensions; there is no extension registry, no community extension store, and no in-display LLM authoring agent. It exposes no HTTP or Ollama-shaped service API: MCP is the interface. It has one rendering mode (native ImGui), not several. It does not host local model inference, does not render to the web or Electron, and does not target mobile or tablet. Applications and applets (clocks, calculators, standalone viewers) are out of scope. These exclusions are what keep the built-in vocabulary coherent and the trust model simple; the full list is in the Feature Appendix.

Risk Assessment

Risk	Rating	Assessment
Value	High	All demonstrated value is *internal*: the current users are one Punt Labs developer plus his agent fleet (see FAQ 8), not the target customer the product is built for: external developers running several terminal agents. "Runs in every session" counts agent sessions, not distinct humans; external adoption is zero. Validation confined to the company that built Lux is a High Value-risk signal in the four-risks framework, so this row is rated High, not Medium. It is why FAQ 10 makes the day-7 external trial the single most important next step.
Usability	Medium	Install is one command and the display opens automatically on first output, so the intended onboarding is simple. But that flow is proven *internally* only; it has never completed on a machine outside Punt Labs. Real prerequisites add friction: Python 3.13+, macOS/Linux only, and a curl-pipe-to-sh install that asks the user to trust a script. The day-7 external cohort (see FAQ 10) is the first real measurement of onboarding-completion; until then the Low-friction claim is a design intent, not a measured result.
Feasibility	Medium	The core is shipped and works: Hub/Display, MCP front door, the data-display core, the interaction loop, and a formally-verified singleton lifecycle. The open work is the v1-to-v2 element migration: 21 of 25 kinds still on the legacy path, with a handler-wrapping seam refactor and the table kind as the hard cases. No single step is high-risk, but the aggregate is substantial and is the primary feasibility exposure. Mitigated by incremental crossing with the legacy path deleted last (see FAQ 14).
Viability	Medium	MIT-licensed with no direct revenue. Strategic value depends entirely on ecosystem pull for the companion Punt Labs tools. If Lux does not drive adoption of biff, vox, and quarry, it is a public good without business return. Acceptable only if treated as infrastructure, which is the stated intent (see FAQ 17).

Feature Appendix

Defines the scope boundary for Lux. Must Do is the shipped v1 identity; Should Do is the v2 direction; Won't Do is the firm exclusion set that keeps the built-in vocabulary coherent and the trust model simple.

Must Do

The core that makes Lux what it is. These ship in v1.

- **F1.** **Typed JSON element protocol**: agents describe UI as a tree of typed elements: tables, text, plots, groups, buttons, and more; the protocol is the API surface
- **F2.** **Native GPU-accelerated rendering**: a persistent, shared ImGui window painting every frame on the developer's own GPU, with no web stack and no vendor compute
- **F3.** **Hub/Display io-model**: `luxd` holds authoritative UI state; the display renders a replica; UI state crosses the wire, render calls do not
- **F4.** **Data-display core**: filterable, searchable tables with row selection and detail panels; dashboards; diagrams; images; 25 element kinds in all
- **F5.** **Ask-the-user interaction loop**: an agent poses a question through a dialog, checkbox, or button and receives the answer back via Hub-side handler dispatch
- **F6.** **Two-tier remote handler dispatch**: the Hub owns handlers; the Display forwards interactions back and the real handler runs on the Hub's copy
- **F7.** **Multi-agent shared window**: multiple agents share one window, each in its own frame with clear attribution
- **F8.** **MCP front door**: a first-class entry point where `mcp-proxy` bridges Claude Code stdio to the `luxd` WebSocket
- **F9.** **Introspection and control surface**: `inspect_scene`, `list_scenes`, `screenshot`, and related tools let an agent verify what actually rendered
- **F10.** **One-command install with auto-open display**: a single curl command installs the package and plugin; the window opens on first output
- **F11.** **Formally-verified singleton lifecycle**: the spawn/reap/bind concurrency (the display singleton lifecycle only, not rendering, protocol, or dispatch) is specified in Z and ProB-model-checked for singleton-serving, never-unlink-live, no-two-winners, clean-exit, and deadlock-freedom

Should Do

The v2 direction. Valuable, not v1-blocking.

- **F12.** **Complete ImGui element coverage**: every ImGui widget available as a curated, built-in element; the point is not coverage for its own sake but the input primitives that enable rich GUI-driven interaction with the agent
- **F13.** **Element-ABC migration**: move the remaining 21 legacy element kinds onto the distributed Element-ABC / Hub-Display path, retiring the legacy render dispatch when the last kind crosses
- **F14.** **Texture-cache eviction policy**: bound the image texture cache, which today grows without eviction
- **F15.** **Companion-tool surfacing**: render biff presence panels, vox controls, and quarry search as first-class Lux views alongside the beads board

Won't Do

Explicitly excluded to keep the vocabulary curated and the trust model simple.

- **F16.** **Self-extending extension registry**: agents will not author and register arbitrary display extensions at runtime; the fixed, curated element set already composes without limit, so infinite element kinds are unnecessary, and Lux stays a built-in vocabulary rather than a plugin platform
- **F17.** **Community extension store**: no marketplace of third-party extensions with ratings and verification badges, which sit outside the trust model Lux commits to
- **F18.** **In-display LLM authoring agent**: Lux does not embed a model that writes new UI code inside the display
- **F19.** **HTTP or Ollama-shaped service API**: MCP is the interface; Lux does not expose a general HTTP service surface
- **F20.** **Multiple rendering modes**: one native ImGui mode only, with no texture-buffer or separate-OS-window rendering modes
- **F21.** **Applications and applets**: clocks, calculators, docks, and standalone viewers are out of scope
- **F22.** **Local model inference**: Lux renders; it does not host models
- **F23.** **Web or Electron rendering**: Lux renders natively via ImGui on the developer's GPU. HTML, Electron, and webviews are out of scope, since web rendering leads to the vendor-compute dependency Lux exists to avoid
- **F24.** **Mobile or tablet**: desktop only (macOS, Linux)

References

[1] Anthropic. *Claude Code*. Agentic coding tool for the terminal. 2025. URL: https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview.

[2] OpenAI. *Codex CLI*. Terminal-hosted agentic coding tool. 2025. URL: https://github.com/openai/codex.

[3] Omar Cornut. *Dear ImGui*. Immediate-mode GUI library for C++. 2025. URL: https://github.com/ocornut/imgui.